Shashank Sekhar *VS*

AI/ML Engineer / Full-Stack Developer / Product Designer

I build systems that reason, ship the products that run them, and design the interfaces people actually touch. Reinforcement-learning agents, production web platforms, and research-backed UX, under one roof.

01 — ABOUT

Engineering graduate working across three disciplines that are usually kept apart: applied AI, production web engineering, and UX research. Currently freelancing as a web developer while leading both the technical and design tracks of my college's department club.

My AI work leans toward systems that don't just predict but act, combining forecasting models with reinforcement-learning agents that make operational decisions. My web work favours Next.js and Supabase, built for real tenants and real traffic, not demos. My design work starts from primary research: surveys, interviews, and usability testing, before a single screen gets drawn.

But more than any one discipline, what I love is building: taking a rough idea and pushing it all the way to something real that people can open, click through, and actually use. That moment when a feature finally works exactly as imagined never gets old, and it's the reason I keep choosing projects that ship rather than projects that stay theoretical.

02 — SELECTED WORK

Things I've shipped

AI Engineering

01

Hybrid AI Framework for Predictive & Adaptive Supply Chain Management

A hybrid system combining CatBoost regression with a PPO reinforcement-learning agent to run e-commerce supply chain operations. CatBoost forecasts demand, profit, delivery time and late-delivery risk; those forecasts become state inputs for an RL agent that adaptively controls inventory, order prioritization, and delivery scheduling via reward-based feedback. Evaluated on the DataCo Smart Supply Chain dataset (~180K records): mean reward of +222.24 vs. −120.5 for the baseline, with strong RMSE, MAE and R^2 across every prediction task.

CatBoost · PPO / RL · Python · Time-series

02

FlowForge — Autonomous Task Decomposition & Execution Engine

A multi-agent pipeline that ingests a project brief and autonomously plans, researches, drafts and reviews deliverables through role-specialized agents with inter-agent feedback loops. Built with CrewAI and the Anthropic API, with quality-threshold exit conditions and structured output to a finished document.

CrewAI · Anthropic API · Multi-agent

03

Trackr — Agentic Health Tracking & Nutritional Intelligence

An AI agent built on LangChain and the Anthropic API that lets users log workouts and meals in natural language, with tool-calling integrations for real-time nutritional lookup and progress analytics. Persistent memory layer for longitudinal tracking and personalized recommendations.

LangChain · Anthropic API · Tool-calling

Web Development

01

Cycle World Pondicherry — Headless Cycle Storefront

Built on Next.js with server-side rendering for SEO-optimized catalogue pages and dynamic product routing. Backed by a normalized Supabase Postgres schema with RLS-enforced access control and Realtime inventory sync, deployed through a Vercel pipeline with edge caching achieving sub-second TTFB.

Next.js · Supabase · Postgres RLS · Vercel Edge

02

QLess — Multi-Tenant Real-Time Pre-Order & Queue Management Platform

A production-deployed SaaS platform where any food vendor can onboard, publish menus and manage orders through a dedicated staff dashboard. Built on Next.js and Supabase with Realtime-powered live token broadcasting, QR-based customer access, and Postgres RLS-enforced per-tenant data isolation.

Next.js | Supabase Realtime | Multi-tenant | QR access

Design

01

Frabe — Women's Personal Safety App

End-to-end Android safety application, driven by primary research across 80 survey respondents and 12 in-depth interviews. Solved the core usability challenge of reaching emergency help in under 8 seconds through a one-tap SOS flow, horizontally scrollable helpline cards, and a safe-area map surfacing 9+ location categories. High-fidelity screens delivered in Figma, with the interaction model stress-tested through Maze usability testing.

User research | Figma | Usability testing

02

QLess — Campus Pre-Order & Queue Management App

Mobile pre-ordering and live token tracking experience for high-footfall campus dining venues, removing physical queues through a streamlined order-to-token flow. Research across canteen visitors, full journey mapping from browse to collection, and 10 high-fidelity screens, built around a sub-3-tap ordering principle.

User research | Journey mapping | Figma

03 — KNOWN TECHNOLOGIES

What I build with

LANGUAGES

- C++
- C
- Python
- Java

FRONTEND

- HTML5
- CSS3
- JavaScript
- Tailwind CSS
- Next.js
- React

BACKEND & APIS

- Node.js
- FastAPI
- Flask

DATABASES

- PostgreSQL / SQL
- MongoDB
- MySQL
- SQLite
- Supabase
- Firebase

AI & ML

- LangChain
- OpenAI
- Anthropic
- Hugging Face

DEVOPS & TOOLS

- Docker
- Kubernetes
- Git
- GitHub
- Vercel
- Figma

---

04 — EXPERIENCE

Where I've worked

Jan 2026 — May 2026

Freelance Web Developer

Built a full-scale webpage end-to-end and handled its SEO.

Jan 2025 — Dec 2026

Technical Lead — College Department Club

Led the technical team across club projects: code quality, system design, and mentoring junior developers in frontend and best practices.

Jan 2025 — Dec 2026

Design Lead — College Department Club

Led the design team's UI/UX initiatives, mentored junior designers in design thinking and prototyping, and ran workshops on user-centered problem solving.

05 — EDUCATION

Where I've studied

2022 — 2026

Bachelor of Technology

Sri Manakula Vinayagar Engineering College

2021 — 2022

12th Grade

Petit Seminaire Higher Secondary School

2019 — 2020

10th Grade

Petit Seminaire Higher Secondary School

SKILLS

WEB

HTML · Tailwind · JavaScript · Next.js

AI / ML

Python · RAG · LangChain

CORE

Java · SQL

06 — CONTACT

Let's talk.

EMAIL
vsshashank08@gmail.com

LINKEDIN
shashank-sekhar

GITHUB
hank0811

PHONE
+91 91502 33560

AI ENGINEERING — CASE STUDY

Hybrid AI Framework for Predictive & Adaptive Supply Chain Management

A system that doesn't just forecast supply chain risk, it acts on it. CatBoost regression models predict demand, profit, delivery time and late-delivery risk; a PPO reinforcement-learning agent turns those forecasts into inventory, prioritization, and scheduling decisions.

ROLE	TYPE	DATASET
AI / ML Engineer	Research + Applied ML	DataCo Smart Supply Chain (~180K records)

CORE RESULT

+222.24 mean reward

01 — OVERVIEW

Forecasting alone isn't a decision

Most supply chain ML stops at prediction. This project treats forecasts as the beginning of the pipeline, not the end, feeding them into an agent that has to actually choose what to do.

E-commerce supply chains fail in ways that are individually predictable but collectively hard to act on: demand spikes, margin-losing SKUs, slow lanes, and orders that are already going to be late. A forecasting model can flag all of this, but someone still has to decide how to prioritize orders, allocate inventory, and schedule delivery under those conditions, continuously, at scale. That's the gap this framework closes.

> ### The problem
>
> Static forecasting models tell you what's likely to happen, but leave the operational response, what to restock, what to prioritize, what to reroute, to manual rules or human judgment that doesn't scale with order volume.

> ### The approach
>
> Split the problem in two: let a strong supervised model (CatBoost) handle what it's good at, accurate multi-metric forecasting, and let a reinforcement-learning agent (PPO) handle what forecasting can't: sequential, reward-driven operational decisions.

> ### The result
>
> An agent that outperforms a static baseline by a wide margin on cumulative reward, while the forecasting layer underneath maintains strong accuracy across every target metric it predicts.

02 — ARCHITECTURE

How the two models talk to each other

The system runs as a two-stage pipeline. Stage one is purely predictive; stage two is purely decision-making, and the handoff between them is the actual engineering problem.

Stage 1 — CatBoost forecasting layer

Four CatBoost regressors trained on historical order data forecast: sales demand, expected profit, delivery time, and late-delivery risk per order/SKU. Chosen for its native handling of categorical features without heavy preprocessing, and strong out-of-the-box accuracy on tabular data.

Stage 2 — PPO decision agent

The four forecasts become part of the state vector for a Proximal Policy Optimization agent. The action space covers inventory control, order prioritization, and delivery scheduling; a shaped reward function scores outcomes on service level, cost, and delay reduction rather than forecast accuracy alone.

Why PPO

PPO was chosen over value-based methods (e.g. DQN) for stability on a mixed discrete/continuous action space and its clipped objective, which keeps policy updates from destabilizing training when the reward landscape is noisy.

03 — TECH STACK

What it's built with

MODELING

CatBoost | PPO (Reinforcement Learning) | scikit-learn

LANGUAGE

Python

DATA

DataCo Smart Supply Chain dataset | Pandas / NumPy

EVALUATION

RMSE | MAE | R^2 | Cumulative reward

04 — RESULTS

Measured against a static baseline

Evaluated on the DataCo Smart Supply Chain dataset (~180,000 records), comparing the full hybrid pipeline against a non-adaptive baseline policy.

+222.24	−120.5
mean reward, RL agent	mean reward, static baseline
~180K	4
records evaluated (DataCo dataset)	forecast targets: demand, profit, delivery time, late-risk

Beyond the headline reward gap, the CatBoost forecasting layer held strong RMSE, MAE and R^2 scores across all four prediction tasks, meaning the agent was making decisions on genuinely reliable state information, not noisy guesses.

05 — LEARNINGS

01

Reward shaping is the real design work

Getting the RL agent to behave sensibly had far less to do with model architecture and far more to do with how the reward function balanced competing goals.

02

Forecast quality bounds decision quality

An RL agent is only as good as the state it's given. Investing early in strong CatBoost forecasts paid off directly in agent stability during training.

03

Baselines make or break the story

Without a clear non-adaptive baseline, a reward number alone means very little. The comparison is what proves the adaptive approach earns its complexity.

Case Study 2 of 7

AI ENGINEERING — CASE STUDY

FlowForge — Autonomous Task Decomposition & Execution Engine

A multi-agent pipeline that takes a single project brief and turns it into a finished deliverable, planning, researching, drafting and reviewing itself through role-specialized agents with feedback loops between them.

ROLE	TYPE	FRAMEWORK	OUTPUT
AI / ML Engineer	Multi-agent system	CrewAI + Anthropic API	Structured document

01 — OVERVIEW

One brief in, a finished draft out

FlowForge exists to answer a specific question: can a brief alone, no step-by-step instructions, be turned into a reviewed, structured deliverable by agents that plan and critique their own work?

A single generalist LLM call struggles with multi-stage work: it tends to blend planning, research and writing into one pass, with no real checkpoint for quality. FlowForge instead decomposes the brief into a pipeline of specialized roles, each with a narrow job and a clear handoff to the next, so the system can catch its own mistakes before they reach the final output.

02 — HOW IT WORKS

Role-specialized agents, inter-agent feedback

> ### Planner
> Ingests the raw project brief and decomposes it into a task graph, discrete, ordered sub-tasks each subsequent agent can act on independently.

> ### Researcher
> Gathers the supporting information each sub-task needs before drafting starts, keeping the writer grounded rather than improvising from the brief alone.

> ### Drafter
> Produces the actual deliverable content per sub-task, using the planner's structure and the researcher's material as its working context.

> ### Reviewer
> Scores the draft against a quality threshold and either approves it or sends it back with specific feedback, the loop that keeps the pipeline from shipping a first draft.

The reviewer's feedback loop is the core design decision: rather than a single linear pass, drafts cycle back to the drafting stage until they clear a defined quality threshold, at which point the pipeline exits and assembles the final structured document.

03 — TECH STACK

What it's built with

ORCHESTRATION

CrewAI · Role-based agents

MODEL

Anthropic API

LANGUAGE

Python

OUTPUT

Structured document export

Quality-threshold exit conditions

04 — LEARNINGS

01

Specialization beats a single mega-prompt

Splitting planning, research, drafting and review into distinct agents produced more coherent output than asking one agent to do all four in sequence.

02

The feedback loop is the product

The reviewer-to-drafter loop, not any individual agent, is what made output quality consistent, the exit condition is what stops the system from either under- or over-iterating.

03

Structured handoffs prevent context loss

Passing structured intermediate outputs between agents, rather than raw free text, kept later stages from losing the intent of earlier ones.

Case Study 3 of 7

AI ENGINEERING — CASE STUDY

Trackr — Agentic Health Tracking & Nutritional Intelligence

An AI agent that lets you log a workout or a meal the way you'd tell a friend about it, then remembers it, looks up the nutrition, and tracks your progress over time.

ROLE	TYPE	FRAMEWORK	MEMORY
AI / ML Engineer	Conversational agent	LangChain + Anthropic API	Persistent, longitudinal

01 — OVERVIEW

Logging health data shouldn't feel like data entry

Most fitness and nutrition apps ask users to fill in structured forms, pick a food from a database, enter grams, select an exercise from a list. Trackr instead takes natural language: “had two eggs and toast, then ran 5k”, and does the structuring itself.

The harder problem isn't parsing that sentence once, it's turning a stream of casual, inconsistent daily entries into a reliable longitudinal record the user (and the agent) can actually reason about weeks later.

02 — HOW IT WORKS

Tool-calling + persistent memory

Natural-language logging

Users describe meals and workouts conversationally. The agent extracts structured entries (food items, quantities, exercise type, duration/intensity) from free text.

Tool-calling integrations

Real-time nutritional lookups are handled through tool calls rather than relying on the model's own memorized nutrition data, keeping figures grounded and current.

Persistent memory layer

Logged entries feed a longitudinal store the agent can query, enabling trend-aware answers instead of one-off responses.

That memory layer is also what powers personalized recommendations, because the agent isn't just reacting to the latest message, it's reasoning over the user's actual history.

03 — TECH STACK

What it's built with

ORCHESTRATION

LangChain · Tool-calling

MODEL

Anthropic API

LANGUAGE

Python

DATA

Persistent memory store · Nutritional lookup API

04 — LEARNINGS

01

Grounding beats memorized facts

Routing nutritional lookups through a real tool call, instead of trusting the model's internal knowledge, avoided the plausible-but-wrong numbers that undermine trust.

02

Memory design is a UX decision, not just infra

What the agent chooses to remember and resurface shapes how personalized it feels, this mattered more to perceived quality than raw model capability.

03

Natural language input needs a forgiving parser

Real users describe meals inconsistently. Handling ambiguity gracefully improved logging accuracy.

Case Study 4 of 7

WEB DEVELOPMENT — CASE STUDY

Cycle World Pondicherry — Headless Cycle Storefront

A server-rendered, SEO-optimized storefront for a real local cycle retailer, built headless on Next.js and Supabase, with realtime inventory and edge caching that keeps pages loading in under a second.

ROLE	TYPE	STACK	PERFORMANCE
Full-Stack Developer	Headless e-commerce	Next.js + Supabase	Sub-second TTFB

01 — OVERVIEW

A catalogue that needs to rank, and load instantly

Cycle World needed a storefront that real customers would find through search, not just a digital catalogue, which meant SEO and load speed weren't nice-to-haves, they were the point.

Client-rendered product listings are invisible to search engines and slow on first paint, both fatal for a local retailer competing on discoverability. The build needed server-rendered catalogue pages that ranked, product routing that scaled with inventory changes, and a backend that could keep stock data accurate without manual updates.

02 — ARCHITECTURE

How it's built

Rendering layer

Built on the Next.js Web Router with server-side rendering for catalogue pages, product listings are fully rendered HTML on first request, optimized for search indexing, with dynamic routing per product.

Data layer

A normalized Supabase Postgres schema backs the catalogue, with Row-Level Security policies enforcing exactly which data each request context can access.

Realtime & delivery

Supabase Realtime keeps inventory in sync as stock changes, while a Vercel deployment pipeline with edge caching pushes rendered pages close to the user.

03 — TECH STACK

What it's built with

FRONTEND

Next.js · Web Router / SSR

BACKEND

Supabase · Postgres · Row-Level Security · Realtime

DELIVERY

Vercel · Edge caching

04 — RESULTS

What it achieves

<1s
time-to-first-byte via Vercel edge caching

SSR
fully server-rendered, SEO-optimized catalogue

RLS
row-level access control on every query

Live
realtime inventory sync, no stale stock data

05 — LEARNINGS

01

SEO shapes the architecture, not just the copy

Choosing SSR over client rendering wasn't a performance afterthought, it was the decision that made the storefront discoverable at all.

02

RLS as the default, not an add-on

Designing the Postgres schema with Row-Level Security from the start avoided retrofitting access control onto a schema that wasn't built for it.

03

Edge caching and realtime can coexist

Balancing aggressive edge caching for speed against realtime inventory accuracy meant being deliberate about what's cached versus what's always fetched live.

Case Study 5 of 7

WEB DEVELOPMENT — CASE STUDY

QLess — Multi-Tenant Real-Time Pre-Order & Queue Management Platform

A production-deployed SaaS platform where any food vendor can onboard, publish a menu, and run live order queues, with per-tenant data isolation and realtime token broadcasting built in from day one.

ROLE	TYPE	STACK	STATUS
Full-Stack Developer	Multi-tenant SaaS	Next.js + Supabase Realtime	Production-deployed

01 — OVERVIEW

One platform, any number of vendors

QLess as an engineering project is a platform, not a single app: any food vendor can sign up, publish their own menu, and manage their own live order queue, without vendors seeing each other's data.

The engineering challenge is different from a single-shop app: the same codebase and database need to safely serve many independent vendors at once, each with their own staff dashboard, menu, and customer queue, while customers get instant, no-login access via QR code.

02 — ARCHITECTURE

How it's built

Multi-tenancy

Postgres Row-Level Security policies enforce per-tenant data isolation at the database layer, so a vendor's menu, orders and staff accounts are structurally unreachable by any other tenant's queries.

Realtime queue & tokens

Supabase Realtime powers live token broadcasting, as orders move through the queue, both the staff dashboard and customer-facing token display update instantly.

Customer access

Customers reach their order and live token status via a QR-based access flow, no account creation, no app install, just a scan.

03 — TECH STACK

What it's built with

FRONTEND

Next.js · Staff dashboard UI

BACKEND

Supabase · Postgres RLS · Realtime

ACCESS

QR-based customer access · Multi-tenant onboarding

04 — RESULTS

Live
production-deployed, real vendors onboarded

N-tenant
any vendor can self-onboard independently

RLS
enforced per-tenant isolation at the database layer

QR
zero-login customer access

05 — LEARNINGS

01

Multi-tenancy has to be a schema decision, not a query filter

Enforcing isolation through RLS at the database level, rather than trusting application code to filter correctly, was the difference between probably safe and actually safe.

02

Realtime is a UX feature disguised as infra

Live token broadcasting wasn't just a technical nice-to-have, it directly created the no-queue feeling that makes the product worth using.

03

Frictionless access drives adoption

Removing account creation entirely in favor of QR-based access measurably lowered the barrier for first-time customers to actually use the queue system.

Case Study 6 of 7

UX/UI CASE STUDY

Frabe — Women's Personal Safety App

A personal safety application that puts emergency response and safe navigation at the tip of your fingers, designed around a single, unforgiving benchmark: help in under 8 seconds.

ROLE	PLATFORM	DURATION	YEAR
UX / UI Design	Android Mobile App	8 Weeks	2025

01 — OVERVIEW

What is Frabe?

Frabe is a personal safety mobile app designed to give women fast, reliable access to emergency services, helplines and safe areas, all in a single tap.

The app bridges the gap between feeling unsafe and getting real help: a one-touch SOS button, horizontally scrolling emergency helplines (Police 100, Women Helpline 1091, Railway Help 182), safe-area maps surfacing nearby hospitals, police stations and pharmacies, CCTV activity logs, and real-time crowd-density monitoring.

> ### My role
>
> I led the end-to-end UX and UI process, from initial user interviews and competitive analysis through wireframing, prototyping, and final high-fidelity screens, and worked with the dev team on pixel-perfect implementation.

> ### The challenge
>
> Existing solutions were fragmented: calling police meant finding a number, safe-area maps lived in a separate app, and there was no unified incident record.

Tools used

Figma, FigJam, Google Forms, Adobe Illustrator.

1 tap

to send an SOS alert with location to contacts and services

3+

helpline cards in a single scrollable row

9+

safe-area categories surfaced on the map

02 — PROBLEM STATEMENT

> *How might we give women in India a fast, confident way to get help and navigate safely, without fumbling through multiple apps in moments of fear?*

4.45M+

crimes against women registered in India in 2022 — NCRB data

72%

of women surveyed feel unsafe on public transport alone after 8 PM

<8 sec

target time to trigger SOS from unlocking the phone

03 — RESEARCH

Understanding the user

Mixed-methods research: 12 in-depth interviews with women aged 18–45, an 80-respondent survey, and competitive analysis of 6 existing safety apps.

In-depth interviews

12 semi-structured interviews across urban and semi-urban areas. Key themes: fear of public transport, difficulty reaching helplines quickly, unfamiliarity with nearby safe locations.

Survey research

80 respondents across Puducherry, Chennai, and Bengaluru. Age range 18–50. 67% had never used a dedicated safety app.

Competitive analysis

Himmat Plus (Delhi Police) — clunky, city-specific. bSafe — strong features, poor discoverability. Shake2Safety — one trick, no rich context. Google Maps SOS — hidden in settings.

Gap found: no app combined one-tap SOS + helplines + safe-area map in a single, beautiful, stress-tested UI.

Survey findings — what women want most

91%
one-tap emergency SOS

84%
nearby safe places on map

79%
quick-dial helplines

75%
alert trusted contacts

58%
CCTV / incident log

46%
crowd density info

04 — USER PERSONAS

Who we're designing for

Priya Rajendran

College Student · Daily Commuter · Age 22

Takes the bus and local train daily between Puducherry and campus, often returning after 9 PM. Tech-savvy, already uses Google Maps, wishes it had a safety layer.

GOALS

- Get help instantly without unlocking and searching
- Know which roads and stops are safest at night
- Alert her mother when something feels wrong

PAIN POINTS

- Police helpline number not memorised
- Existing apps are slow to open under stress
- Doesn't know nearest hospital in unfamiliar areas

Meena Subramanian

Working Professional · Administrator · Age 38

Manages a small team at a government office, travels to field sites, handles incident reports. Wants a tool her team can use to file and view SOS alerts from one dashboard.

GOALS

- Monitor SOS alert logs and respond quickly
- Review CCTV and crowd-count reports in one place
- Ensure field team members are safe

PAIN POINTS

- Scattered tools for different safety functions
- No historical log to review past incidents
- Difficulty proving incidents without CCTV records

05 — USER JOURNEY

Priya's emergency scenario

STAGE	ACTION	THOUGHTS	EMOTION
1. Sense danger	Notices she's being followed on an empty street	“Something feels wrong. I need help now.”	Panicked
2. Open app	Unlocks phone, taps Frabe icon	“I hope this loads fast.”	Anxious
3. Trigger SOS	Taps the large red SOS button	“Did it go? Is someone coming?”	Tense
4. Call helpline	Swipes to Women Helpline (1091), taps	“I need to speak to someone now.”	Cautious
5. Find safe place	Taps “View Safe Areas”	“Where can I go right now?”	Searching
6. Reach safety	Navigates to pharmacy 200m away	“I'm safe. The alert was sent.”	Relieved

06 — PAIN POINTS

What was broken

01

Too many steps to get help

Average time to reach an emergency call in competitor apps: 14–22 seconds.

02

No integrated safe-location finder

Users had to switch to Google Maps and manually search.

03

Helpline numbers not memorised

72% of surveyed users couldn't recall the Women Helpline (1091) under simulated stress.

04

No incident audit trail

No centralised log of SOS events, CCTV activity, or crowd density.

07 — HOW MIGHT WE

Reframing the problem

HMW 01

How might we reduce the time to trigger an SOS to under 8 seconds from phone unlock?

HMW 02

How might we surface all critical helplines without users memorising a single number?

HMW 03

How might we show nearby safe areas in a single tap, without switching apps?

08 — UI DESIGN

The final screens

Every screen designed around one principle: usable under stress. Large tap targets, high-contrast colours, minimal cognitive load at every step.

SOS Alert — one tap, immediate action. A radial-gradient SOS button with triple pulse rings dominates the screen. Dark-toned helpline cards give each service its own visual identity, with the exact dial number always visible.

Authentication

Custom SVG illustrations reinforce the app's mission before a single tap. A Login/Sign Up tab toggle eliminates navigation confusion; admin login stays accessible but not prominent.

Admin panel

A clean log-selection panel with three log types (SOS, CCTV, Crowd Count), each with iconography and colour-coded categories for quick scanning.

09 — DESIGN SYSTEM

Colours & typography

A constrained palette built for accessibility and emotional clarity, hot pink signals urgency, deep navy grounds authority, soft neutrals provide breathing room.

Primary Pink
#FF0080

Pink Dark
#C8005E

Pink Light
#FFF0F7

SOS Red
#D94030

Deep Navy
#1A1A2E

Surface
#F8F7F5

10 — DESIGN DECISIONS

Why we made these choices

01

Horizontal scrolling helpline cards

Each helpline gets its own colour, icon and number, scannable and memorable even under stress.

02

Bottom sheet for Safe Areas

Keeps the SOS button visible at all times, the user never leaves the emergency context.

03

Illustration on auth screens

Empowered, not victimised, illustrations make the app feel warm and supportive from the very first screen.

04

SOS button sizing and placement

180px diameter with triple pulse rings, the largest tappable element in the app, guided by Fitts's Law.

11 — OUTCOMES

Impact & results

Usability testing with 14 participants across two rounds, plus stakeholder feedback.

6.2s

avg. time app-open to SOS trigger — down from a 14s baseline

94%

task success rate for SOS + finding a safe area

4.7★

average usability rating from 14 test participants

0

navigation errors finding Women Helpline after redesign

12 — LEARNINGS

01

Stress-test your designs, literally

Asking participants to operate the prototype mid-conversation revealed cognitive load standard testing missed, leading to a 30% larger SOS button.

02

Emotional tone is a design element

Illustrated characters transformed reactions, “this feels like it cares about me” was said unprompted by three testers.

03

Admin and user needs diverge sharply

Merging admin logs into the user flow was wrong, admins need density, users need radical simplicity.

Case Study 7 of 7

UX/UI CASE STUDY

QLess — Campus Pre-Order & Queue Management App

A food pre-ordering app that lets you schedule your meal, pay in advance, and collect with a single spoken OTP, zero waiting, zero friction.

ROLE	PLATFORM	DURATION	YEAR
UX / UI Design	Android Mobile App	6 Weeks	2025

01 — OVERVIEW

What is QLess?

QLess is a dedicated mobile pre-ordering app for a single food shop. Schedule a pickup time, pay in advance, and receive a unique OTP, say it at the counter, walk out with food in hand. No queue, no fumbling, no wasted time.

Unlike a marketplace such as Swiggy or Zomato, QLess is built exclusively for one shop, enabling deep integration, zero commission, and a seamless experience for that shop's loyal customers.

> ### The challenge
>
> People skip meals or make poor food choices not for lack of appetite, but because they can't afford the time cost of a queue.

> ### My role
>
> End-to-end UX and UI design, user interviews and research through wireframing, prototyping, and final high-fidelity screens across all 10 app states.

Tools used

Figma, FigJam, Google Forms, Maze.

1

OTP to collect your meal — no card, no queue

10

screens designed end-to-end for the full journey

<30s

target time from app open to completed order

02 — PROBLEM STATEMENT

> *How might we let busy people pre-order food for a specific time, pay in advance, and collect it instantly, without waiting in a single queue?*

12–18

average minutes lost per person per queue visit

63%

skip meals due to time pressure on weekdays

<30s

target time from app open to order placed

03 — RESEARCH

Understanding the user

10 in-depth interviews with working professionals, students and commuters, a 60-respondent survey, and competitive analysis of existing ordering apps.

In-depth interviews

10 semi-structured interviews, ages 18–42. Key themes: frustration with unpredictable wait times, anxiety about being late, desire for a guaranteed ETA.

Survey research

60 respondents across Puducherry and Chennai, ages 18–50. 71% had never pre-ordered food from a local shop before.

Competitive analysis

No single-shop app combined time-slot scheduling + prepayment + OTP pickup. Swiggy/Zomato add delivery overhead; WhatsApp orders have no payment or scheduling integration.

Survey findings — what users want most

88%
pre-schedule pickup time

81%
pay in advance, no cash at counter

76%
real-time slot availability

73%
simple OTP-based pickup

58%
order history and reorder

44%
push notification reminders

04 — USER PERSONAS

Who we're designing for

Karthik Selvam

College Student · Daily Commuter · Age 21

GOALS

- Eat a proper meal without being late to class
- Know exactly when food will be ready
- Pay digitally, no cash fumbling

PAIN POINTS

- Canteen queues run 15–20 min at peak hours
- Food often cold or sold out on late arrival
- No existing way to plan ahead

Anitha Krishnan

Working Professional · Office Manager · Age 34

GOALS

- Order lunch before leaving office, collect on arrival
- Use the same shop daily without the queue penalty
- Reorder favourites with minimal taps

PAIN POINTS

- 30-minute lunch break, queue wastes half of it
- No app serves her local shop
- Delivery apps add fee and delay — she's 200m away

05 — USER JOURNEY

Karthik's lunch scenario

STAGE	ACTION	THOUGHTS	EMOTION
1. Decide to eat	Checks time between classes — 40 min gap	“I have time but not if I queue.”	Calculating
2. Open app	Sees dashboard with menu categories	“I need Masala Dosa and a coffee.”	Focused
3. Browse & add	Taps Masala Dosa, adds to cart	“That's what I want. Add it.”	Confident

STAGE	ACTION	THOUGHTS	EMOTION
4. Pick time slot	Sees 11:30 AM slot available	“Perfect, I'll be there right at 11:30.”	Relieved
5. Pay & confirm	Reviews summary, pays via UPI	“Done. I can stop thinking about this.”	Satisfied
6. Collect	Says “7421” at the shop, gets food in 10 sec	“So much better than queuing.”	Delighted

06 — PAIN POINTS

What was broken

01

No way to predict wait time

Every visit was a gamble against the clock, no signal of kitchen load or wait time.

02

No single-shop pre-order app

Marketplaces don't serve small shops; WhatsApp ordering has no payment or pickup-time confirmation.

03

Cash-only at the counter

64% preferred digital payment, but shops had no integration.

04

Food sold out or cold on arrival

Popular items sold out by peak hours for anyone arriving without pre-ordering.

07 — HOW MIGHT WE

Reframing the problem

HMW 01

How might we let users place an order and lock in a pickup time in under 30 seconds?

HMW 02

How might we make prepayment feel safe and instant so users never hesitate at checkout?

HMW 03

How might we make pickup frictionless so a 4-digit code replaces every counter interaction?

08 — UI DESIGN

The final screens

Designed around one principle: the fastest path from hungry to fed. Large tap targets, clear hierarchy, and a calm two-tone palette at every step.

Onboarding that feels fast, not clinical. A toggle between Login and Sign Up eliminates navigation confusion. A deep forest-green header creates instant brand recognition; an amber CTA stays unmissable against the neutral form background.

Dashboard — browse like Swiggy, order for your shop. A fixed header with location and search, horizontal category chips, and a vertical food list grouped by category.

OTP pickup — confirmed in seconds. A unique 4-digit OTP in large, high-contrast digits with a countdown timer directly below.

09 — DESIGN SYSTEM

Colours & typography

A constrained two-tone palette, deep forest green for trust and authority, warm amber for action and energy, with neutral surfaces providing breathing room.

Deep Forest
#1A3C34

Forest Mid
#2E6B5E

Forest Light
#E8F2EF

Amber CTA
#F4A227

Amber Dark
#7A4A00

Surface
#F8F7F5

10 — DESIGN DECISIONS

Why we made these choices

01

Spoken OTP over QR code

Faster than scanning, no brightness issues, no QR app, no awkward phone-to-counter angle.

02

Prepayment as a benefit, not friction

Reframed as “your food is waiting, fully paid”, removing the cash-fumbling interaction entirely.

03

Swiggy-style dashboard for familiarity

Borrowing a mental model users already had reduced learning time to zero.

04

Countdown timer below OTP

Resolved “has it expired?” anxiety discovered in testing.

11 — OUTCOMES

Impact & results

Usability testing with 12 participants across two rounds, plus stakeholder review with the shop owner.

22s

avg. time app-open to order placed — vs. a 30s target

96%

task success rate for the full order flow

4.8★

average usability rating from 12 test participants

0

navigation errors on the OTP screen after round 2

12 — LEARNINGS

01

Familiarity is a design asset, not laziness

Reusing an existing mental model eliminated the learning curve entirely.

02

Anxiety appears in unexpected places

No one anticipated OTP-validity stress. You can only discover anxiety points through testing.

03

Constraints on the kitchen are UX features

Limiting time-slot capacity felt like a product decision, but testing showed it was crucial.